

# Final Exam (part 1) - Computational Physics I

Deadline: Thursday 5 June 2025 (by 23h59)

10/10

Credits: 10 points

Good work!

Please keep the structure provided below and submit an organised notebook with clear answers to each item.

Name: Juan Daniel Vasconez

## 1. Fourier analysis: removing Herringbone MRI artifacts

This problem consist of using Fourier analysis to remove undesired Herringbone artifacts from magnetic resonance images (MRI).

The file provided below **herringbone-mri-artifact.jpg** contains a brain MRI image affected by the Herringbone artifact. This artifact can be produced by electromagnetic spikes by gradient coils, a fluctuating power supply, or radiofrequency pulse discrepancies during MRI scanning. In real space, the regularly spaced stripes resemble the appearance of a fabric with a herringbone pattern (which gives the artifact its name). In this image, the artifact covers the entire image with multiple diagonal stripes, which would be seen as spike artifacts in Fourier space. Your task is to remove them.

Download the image file:

<https://github.com/wbandabarragan/physics-teaching-data/blob/main/2D-data/herringbone-mri-artifact.jpg>

Reference:

Case courtesy of Frank Gaillard, [Radiopaedia.org](https://radiopaedia.org). From the case [rID: 19695](#)

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

Image I/O:

(a) Inspect the file. Then, write a python function that reads the data from the **herringbone-mri-artifact.jpg** file, and returns its 3 layers (channels) as numpy arrays.

(b) Call your function, select a single channel and make a map of it. Identify and label the Herringbone artifacts on the image using small arrows.

```
In [6]: # Define the file path for the MRI image with herringbone artifacts
✓ fname = './herringbone-mri-artifact.jpg'

def image_to_array(fname, channel=0):
    """
    Read an image file and convert it to a numpy array.

    Parameters:
    fname (str): Path to the image file
    channel (int): Channel index to extract (default=0 for red channel)
                  If image is grayscale, returns the entire image

    Returns:
    numpy.ndarray: 2D array representing the selected image channel as float
    """
    # Read the image using matplotlib's imread function
    ✓ img = mpimg.imread(fname)

    # Extract specific channel if image is 3D (RGB), otherwise use entire image
    ✓ image = img[:, :, channel] if img.ndim == 3 else img

    # Convert to float32 numpy array for numerical processing
    ✓ return np.array(image, dtype=np.float32)

# Load the image and extract the first channel (red channel)
✓ image = image_to_array(fname)
print(f"Image array shape: {image.shape}")

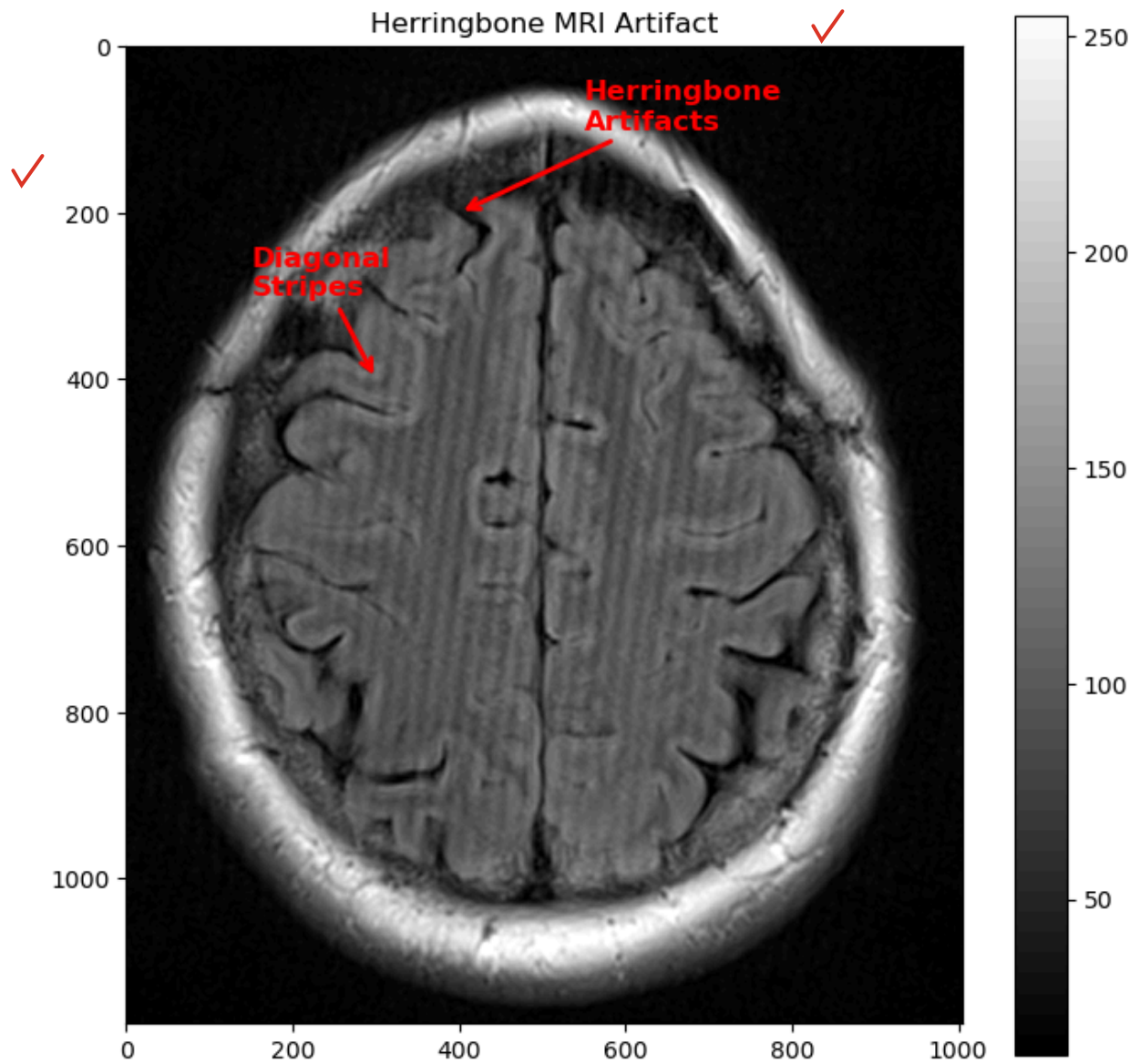
# Display the MRI image with herringbone artifacts
plt.figure(figsize=(8, 8))
z = plt.imshow(image, cmap='gray')
plt.colorbar(z)
plt.title('Herringbone MRI Artifact')

# Add arrows to identify herringbone artifacts
# Arrow pointing to diagonal stripe artifacts in upper region
✓ plt.annotate('Herringbone\nArtifacts', xy=(400, 200), xytext=(550, 100),
              arrowprops=dict(arrowstyle='->', color='red', lw=2),
              fontsize=12, color='red', fontweight='bold')

# Arrow pointing to diagonal stripe artifacts in middle region
✓ plt.annotate('Diagonal\nStripes', xy=(300, 400), xytext=(150, 300),
              arrowprops=dict(arrowstyle='->', color='red', lw=2),
              fontsize=12, color='red', fontweight='bold')

plt.show()
```

Image array shape: (1175, 1006)



## 2D Fourier transform:

(c) Create a function that Fourier transforms this image and returns the Fourier image and the correct frequencies.

(d) Call your function in (c), and make a two-panel figure with the original image on the left and a labeled 2D plot of its Fourier image on the right. Identify and label the Herringbone artifacts on the Fourier image.

(e) Briefly comment, how do the image artifacts look in Fourier space?

In [7]:

```
def fft_transform(image):
    """
    Perform 2D Fast Fourier Transform on an image and return the transformed image and frequencies.

    Parameters:
    image (numpy.ndarray): 2D input image array
```

✓

Returns:

tuple: Contains the following elements:

- transformed\_shifted (numpy.ndarray): FFT-shifted 2D Fourier transform
- fftx (numpy.ndarray): 2D meshgrid of x-direction frequencies
- ffty (numpy.ndarray): 2D meshgrid of y-direction frequencies
- magnitude (numpy.ndarray): Magnitude (absolute value) of the FFT

.....

*# Compute 2D FFT of the input image*

transformed = np.fft.fft2(image)

*# Shift zero frequency component to center for better visualization*

transformed\_shifted = np.fft.fftshift(transformed)

*# Generate frequency arrays for both dimensions*

freqs\_x = np.fft.fftfreq(image.shape[0])

freqs\_y = np.fft.fftfreq(image.shape[1])

*# Create 2D meshgrids of frequencies for plotting*

fftx, ffty = np.meshgrid(sorted(freqs\_x), sorted(freqs\_y), indexing='ij')

*# Calculate magnitude spectrum for visualization*

magnitude = np.abs(transformed\_shifted)

return transformed\_shifted, fftx, ffty, magnitude

*# Get the FFT result*

fft\_transformed\_image, freq\_x, freq\_y, magnitude = fft\_transform(image)

*# Create side-by-side comparison of original image and its FFT*

fig, axs = plt.subplots(1, 2, figsize=(10, 6))

*# Left panel: Display original MRI image*

im1 = axs[0].imshow(image, cmap='gray')

axs[0].set\_title('Original Image')

*# Right panel: Display log-magnitude of FFT to reveal frequency components*

*# Log scale used to compress dynamic range and highlight artifact spikes*

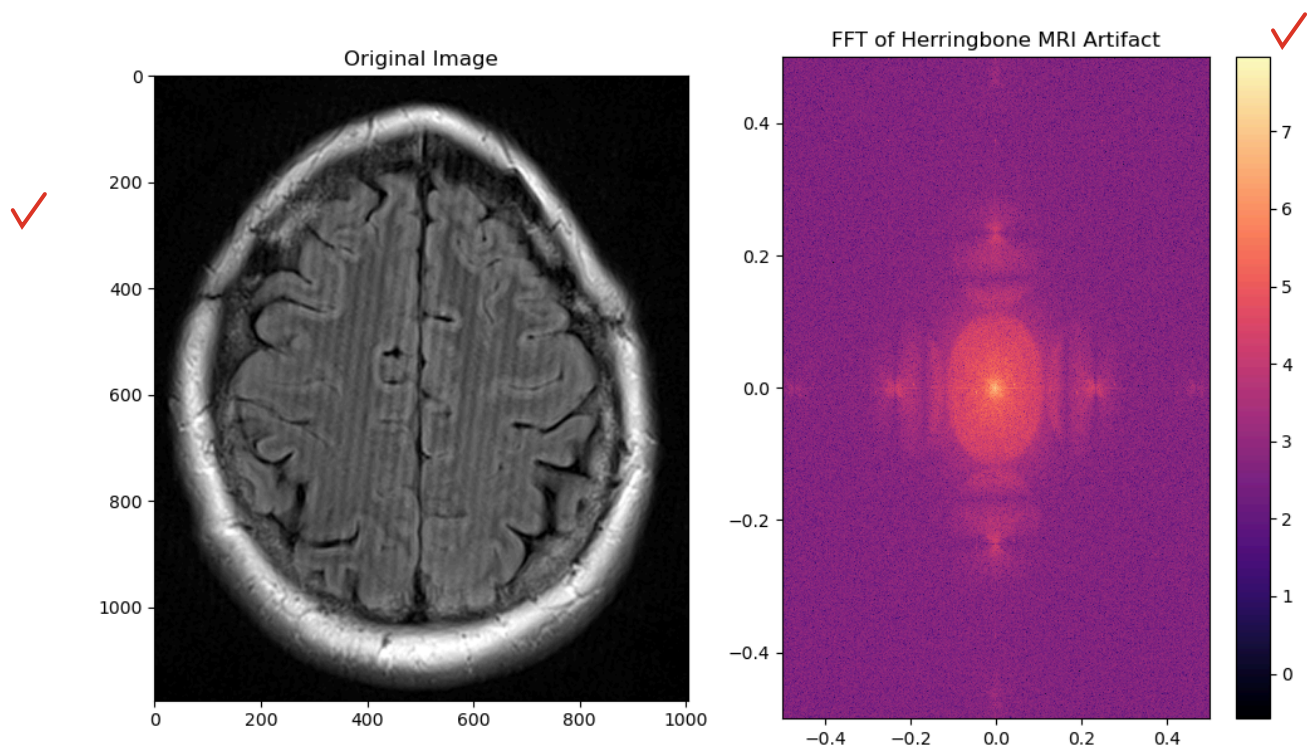
im2 = axs[1].pcolor(freq\_y, freq\_x, np.log10(magnitude), cmap='magma')

axs[1].set\_title('FFT of Herringbone MRI Artifact')

plt.colorbar(im2, ax=axs[1])

plt.tight\_layout()

plt.show()

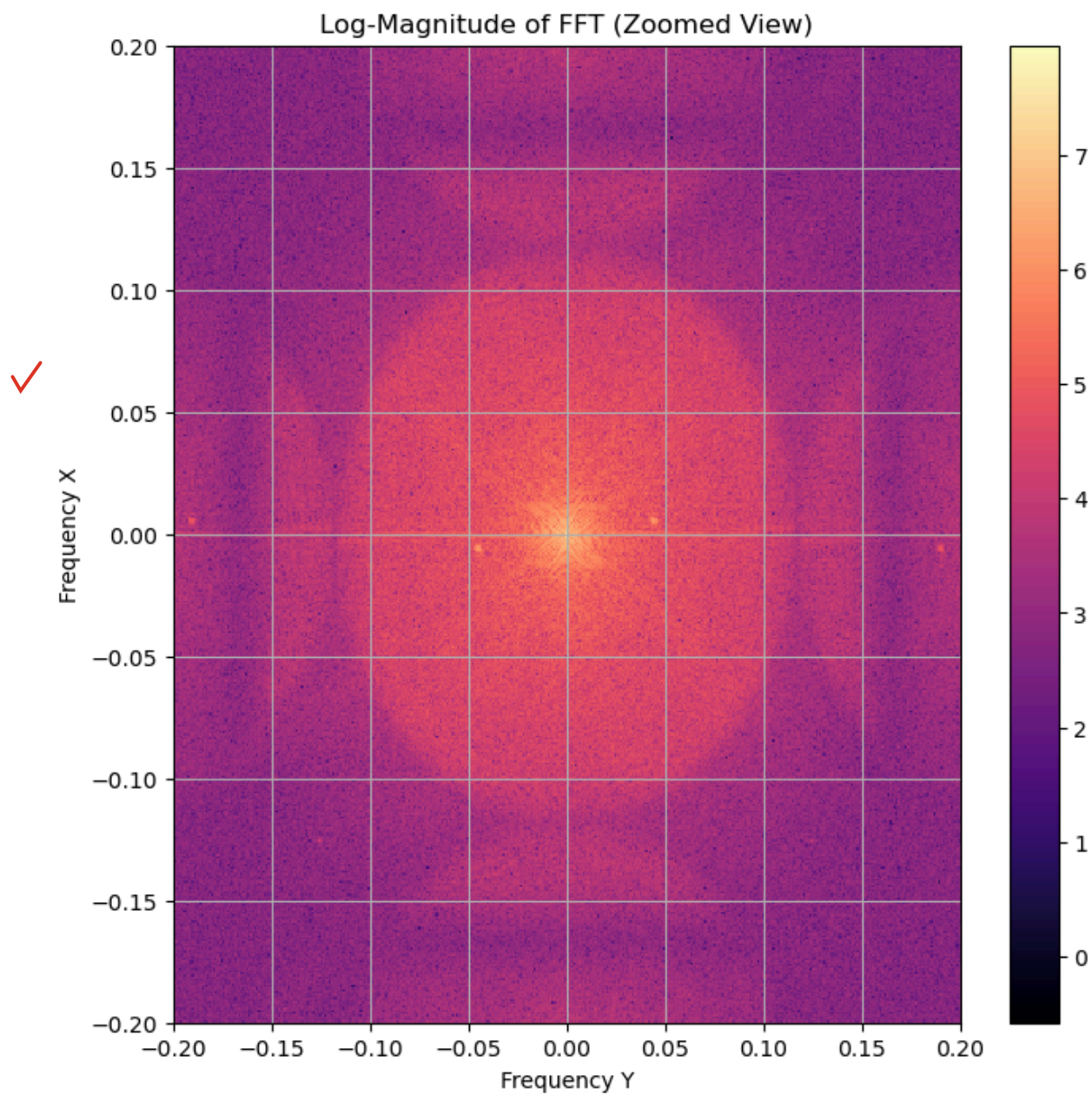


Here on the FFT Transform of the image signal we can notice some really small points that are unusual in the image, so for further analysis let's create a zoomed image for this FFT transform

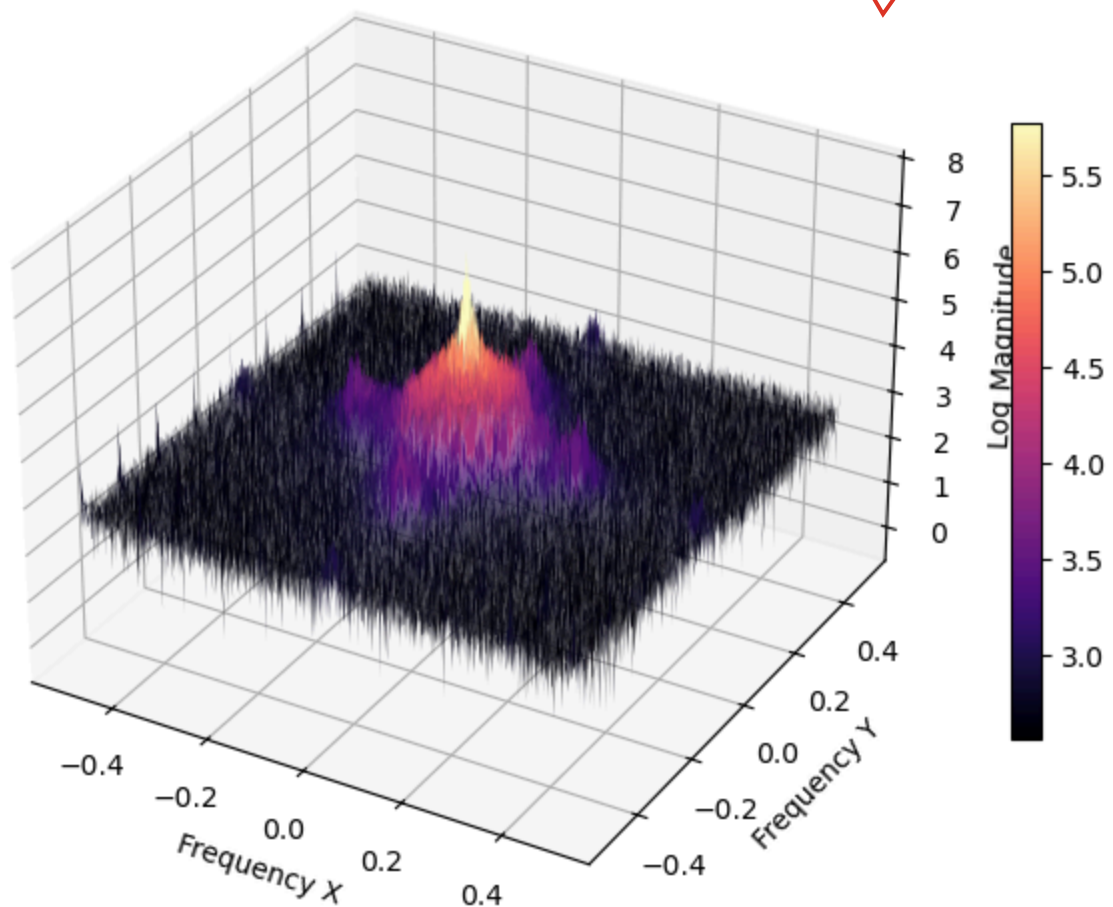
```
In [9]: # Visualize the log-magnitude of the FFT with zoomed view to identify artifacts
plt.figure(figsize=(8, 8))
plt.pcolor(freq_y, freq_x, np.log10(magnitude), cmap='magma')
plt.title('Log-Magnitude of FFT (Zoomed View)')
plt.grid(True)
# Zoom into the frequency range where artifacts are most visible
plt.xlim(-0.2, 0.2)
plt.ylim(-0.2, 0.2)
plt.xlabel('Frequency Y')
plt.ylabel('Frequency X')
plt.colorbar()
plt.show()

# Create a 3D surface plot to better visualize the frequency domain spikes
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection='3d')
surf = ax.plot_surface(freq_x, freq_y, np.log10(magnitude), cmap='magma')
ax.set_title('3D Surface Plot of Log-Magnitude FFT')
ax.set_xlabel('Frequency X')
ax.set_ylabel('Frequency Y')
ax.set_zlabel('Log Magnitude')
# Optional: Uncomment to limit the view to specific ranges
# ax.set_xlim(-0.05, 0.05)
# ax.set_ylim(-0.05, 0.05)
# ax.set_zlim(-5, 5)
plt.colorbar(surf, shrink=0.5)
plt.show()
```





### 3D Surface Plot of Log-Magnitude FFT



We can see that there are unusual peaks in the frequencies domain, which analyzing in the hole image we can see they are concurrent and periodic, exactly as the Herringbone Artifacts. Which are peaks in the frequencies domain

### Masking and filtering:

(f) Create an appropriate mask for the Fourier image that matches the features associated with the Herringbone artifacts. Plot the mask.

(g) Use your mask from (f) to remove the Herringbone artifacts from the Fourier image.

(h) Inverse Fourier transform the masked Fourier image and make a two-panel figure showing the original image on the left and the new filtered image (without the Herringbone artifacts) on the right.

```
In [12]: # Define the locations and radius for the two dots
dot1_x, dot1_y = -0.035, 0.007 # First dot location in frequency space
dot2_x, dot2_y = -0.13, -0.005 # Second dot location in frequency space
dot_radius = 0.005 # Small radius for each dot
```

```

# Create coordinate arrays for the frequency space
ny, nx = image.shape
center_x, center_y = nx//2, ny//2

# Create mask (start with ones - keep everything)
mask = np.ones((ny, nx))

# Convert frequency coordinates to array indices
def freq_to_index(freq_val, size):
    return int(center_y + freq_val * size)    Add docstring.

# Create circular masks for each dot
y_indices, x_indices = np.ogrid[:ny, :nx]    ok.

# First dot
idx1_x = freq_to_index(dot1_x, nx)
idx1_y = freq_to_index(dot1_y, ny)
radius_pixels = int(dot_radius * min(nx, ny))

dot1_mask = (x_indices - idx1_x)**2 + (y_indices - idx1_y)**2 <= radius_pixels

# Second dot
idx2_x = freq_to_index(dot2_x, nx)
idx2_y = freq_to_index(dot2_y, ny)

dot2_mask = (x_indices - idx2_x)**2 + (y_indices - idx2_y)**2 <= radius_pixels

# Set mask to zero at dot locations (remove these frequencies)
mask[dot1_mask] = 0
mask[dot2_mask] = 0

# Visualize the mask
plt.figure(figsize=(8, 8))
z = plt.imshow(mask, cmap='magma')
plt.colorbar(z)
plt.title("Two-Dot Filter Mask")
plt.grid(True)
plt.show()

# Apply mask to FFT
filtered_fft = fft_transformed_image * mask
inv_fft_shifted = np.fft.ifftshift(filtered_fft)
inv_fft = np.fft.ifft2(inv_fft_shifted)

# Get filtered image
filtered_image = np.abs(inv_fft)

# Show filtered FFT
plt.figure(figsize=(8, 8))
z = plt.imshow(np.log10(np.abs(filtered_fft) + 1), cmap='magma')
plt.colorbar(z)
plt.title("Filtered FFT (Two Dots Removed)")
plt.xlim(350, 650)
plt.ylim(450, 725)
plt.grid(True)
plt.show()

```

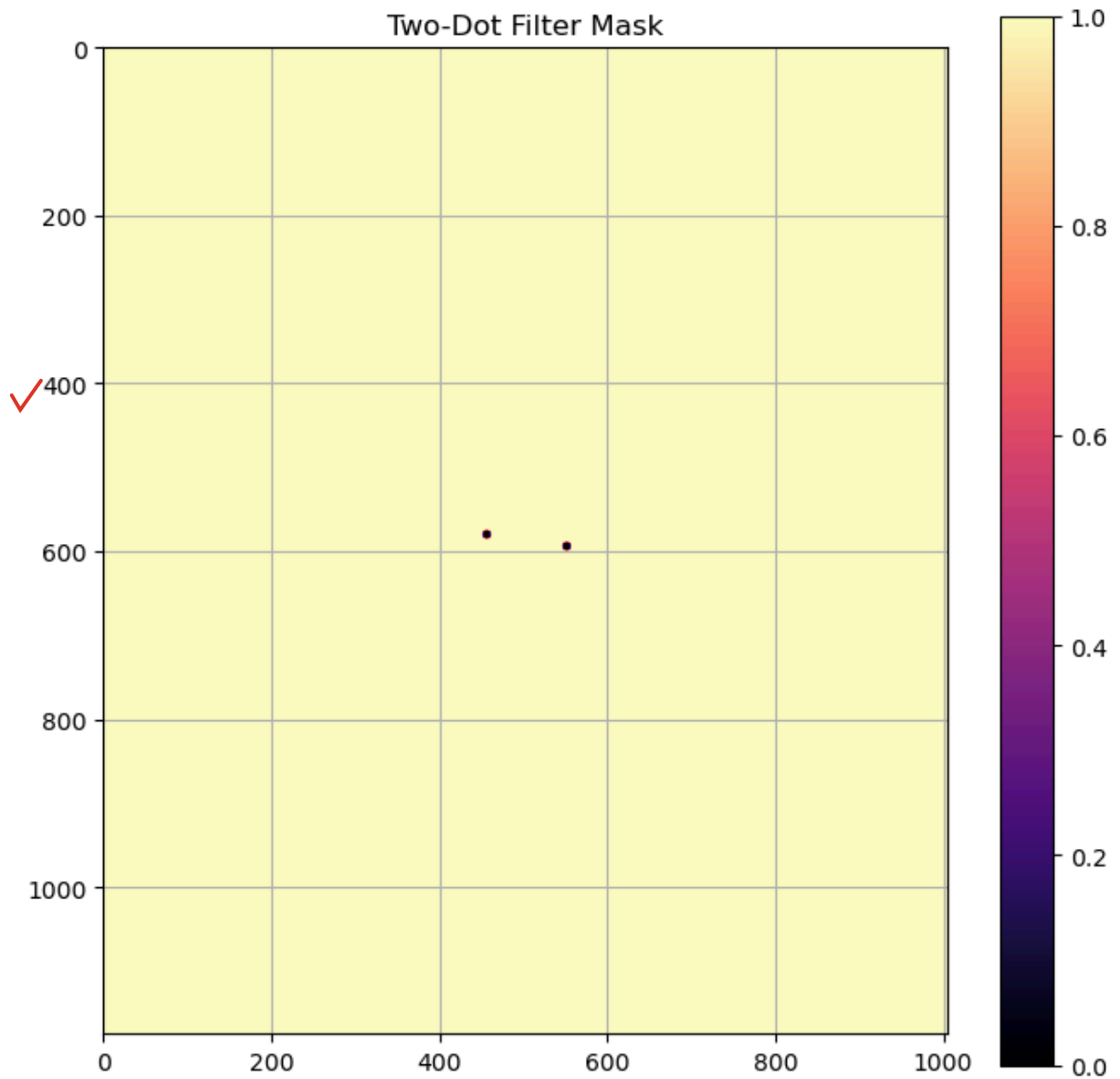


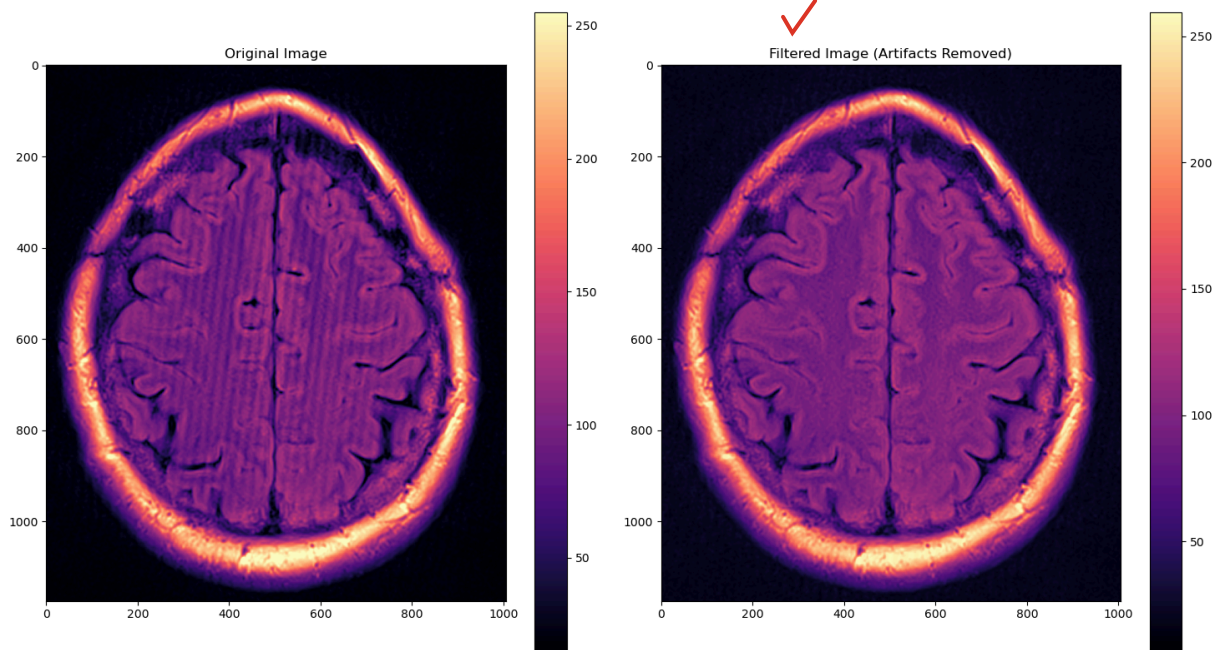
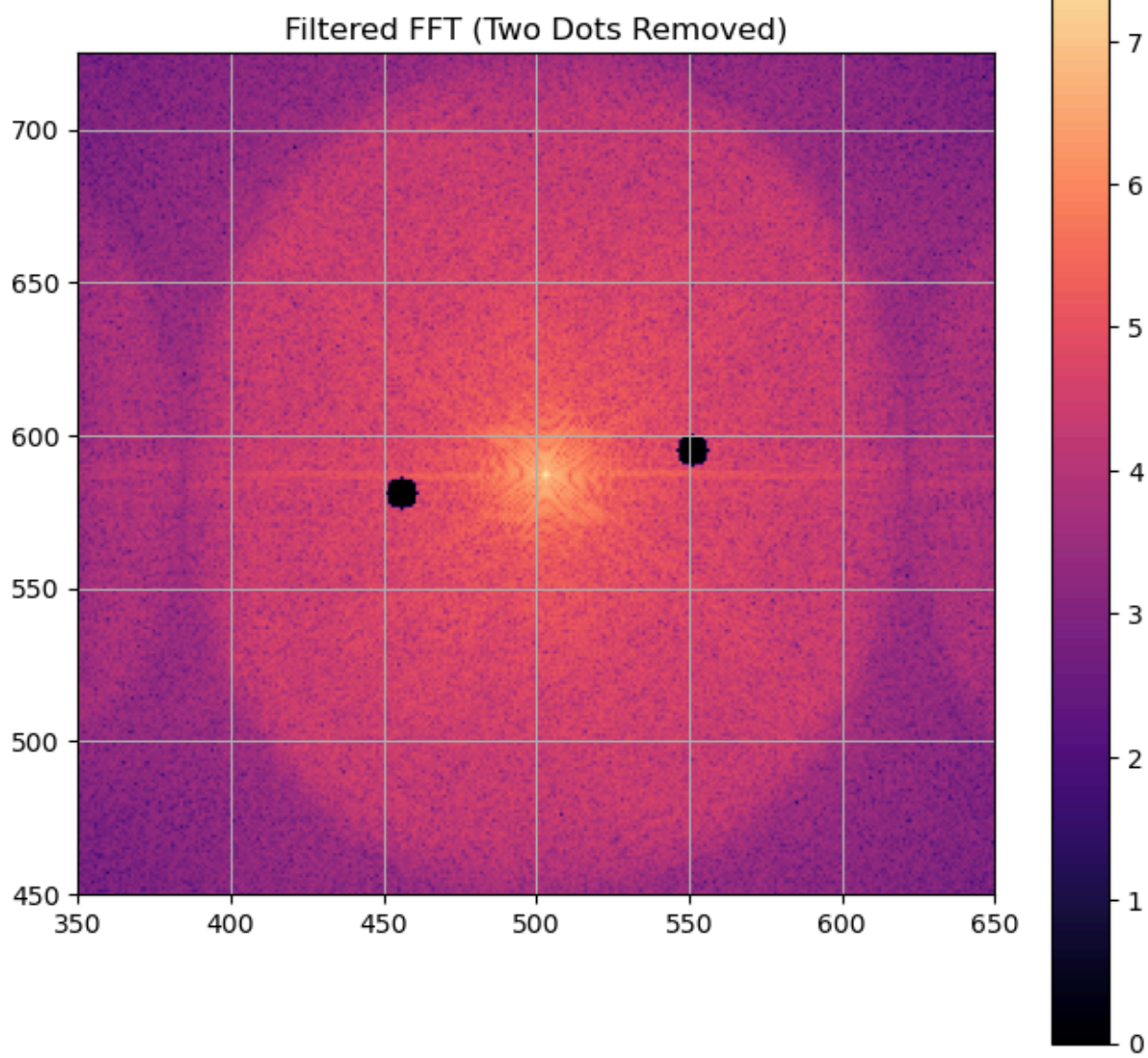
```
# Two-panel comparison
fig, axs = plt.subplots(1, 2, figsize=(15, 8))

ima1 = axs[0].imshow(image, cmap='magma')
axs[0].set_title('Original Image')
plt.colorbar(ima1, ax=axs[0])

ima2 = axs[1].imshow(filtered_image, cmap='magma')
axs[1].set_title('Filtered Image (Artifacts Removed)')
plt.colorbar(ima2, ax=axs[1])

plt.tight_layout()
plt.show()
```





## Artifact-free image:

(i) Create a function that combines all the previous steps. The function should receive an input image channel, apply the Fourier filter to it and return the original and the filtered channel images. Call and apply your filter function to the other two channels of the **herringbone-mri-artifact.jpg** image.

(j) Using the output clean images from (i), reconstruct the 3-layered image and make a two panel figure showing the original 3-layered image and your new artifact-free 3-layered image. Briefly comment on your results.

```
In [13]: def fourier_filter_image(input_image):
        """
        Apply Fourier filtering to remove herringbone artifacts from an image channel

        Parameters:
        input_image: 2D numpy array representing a single image channel

        Returns:
        original_image: The input image
        filtered_image: The filtered image with artifacts removed
        """
        # Get FFT of the image
        fft_transformed = np.fft.fft2(input_image)
        fft_shifted = np.fft.fftshift(fft_transformed)

        # Create the same mask as before
        ny, nx = input_image.shape
        center_x, center_y = nx//2, ny//2

        # Create mask (start with ones - keep everything)
        mask = np.ones((ny, nx))

        # Convert frequency coordinates to array indices
        def freq_to_index(freq_val, size):
            return int(center_y + freq_val * size)

        # Create circular masks for each dot
        y_indices, x_indices = np.ogrid[:ny, :nx]

        # First dot parameters (same as before)
        dot1_x, dot1_y = -0.035, 0.007
        dot2_x, dot2_y = -0.13, -0.005
        dot_radius = 0.005

        # First dot
        idx1_x = freq_to_index(dot1_x, nx)
        idx1_y = freq_to_index(dot1_y, ny)
        radius_pixels = int(dot_radius * min(nx, ny))

        dot1_mask = (x_indices - idx1_x)**2 + (y_indices - idx1_y)**2 <= radius_

        # Second dot
```

```

idx2_x = freq_to_index(dot2_x, nx)
idx2_y = freq_to_index(dot2_y, ny)

dot2_mask = (x_indices - idx2_x)**2 + (y_indices - idx2_y)**2 <= radius_

# Set mask to zero at dot locations (remove these frequencies)
mask[dot1_mask] = 0
mask[dot2_mask] = 0

# Apply mask to FFT
filtered_fft = fft_shifted * mask

# Inverse FFT
inv_fft_shifted = np.fft.ifftshift(filtered_fft)
inv_fft = np.fft.ifft2(inv_fft_shifted)

# Get filtered image
filtered_image = np.abs(inv_fft)

return input_image, filtered_image

# Read all three channels of the original image
img_full = mpimg.imread(fname)

# Apply filter to each channel
channel_0_orig, channel_0_filtered = fourier_filter_image(img_full[:, :, 0])
channel_1_orig, channel_1_filtered = fourier_filter_image(img_full[:, :, 1])
channel_2_orig, channel_2_filtered = fourier_filter_image(img_full[:, :, 2])

# Reconstruct the 3-layered images
original_3layer = np.stack([channel_0_orig, channel_1_orig, channel_2_orig],
filtered_3layer = np.stack([channel_0_filtered, channel_1_filtered, channel_

# Normalize the filtered image to proper range
filtered_3layer = np.clip(filtered_3layer / 255.0, 0, 1)
original_3layer = original_3layer / 255.0

# Two-panel comparison
fig, axs = plt.subplots(1, 2, figsize=(15, 8))

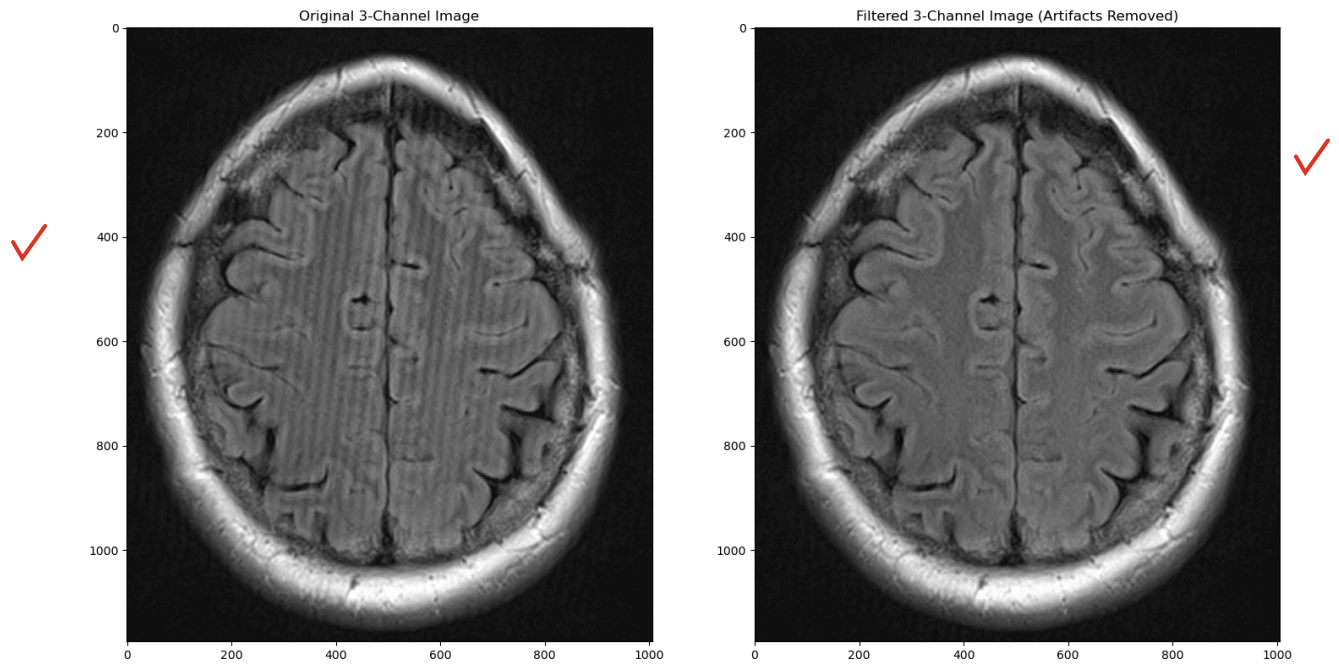
axs[0].imshow(original_3layer, cmap='magma')
axs[0].set_title('Original 3-Channel Image')

axs[1].imshow(filtered_3layer, cmap='magma')
axs[1].set_title('Filtered 3-Channel Image (Artifacts Removed)')

plt.tight_layout()
plt.show()

print("Filter successfully applied to all three channels.")
print(f"Original image shape: {original_3layer.shape}")
print(f"Filtered image shape: {filtered_3layer.shape}")

```



Filter successfully applied to all three channels.

Original image shape: (1175, 1006, 3)

Filtered image shape: (1175, 1006, 3)

Results: The Fourier filtering successfully removed the Herringbone artifacts from all three channels. The diagonal stripe patterns visible in the original image have been eliminated, resulting in a cleaner MRI image with preserved anatomical details.